Heart Disease Prediction with Multiple Metrics

ML PROJECT

INTRODUCTION:

This project predicts heart disease using machine learning techniques.It helps in early detection and analysis.

DATASET:

Dataset: Heart.csv
Source: Kaggle
Contains medical attributes and target variable.

1. Model Selection:

Problem Type: Classification

Algorithms Used:

1) Logistic Regression

2) KNN

Reason: Suitable for classification problems.

2. Data Preprocessing:

- **<u>Import the dataset:</u>**

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import  accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import r2_score

df=pd.read_csv("heart.csv")
```

- **<u>Display basic information (head, info):</u>**

```
df.head()
df.tail()
```

```
[6]: df.head()
```

[6]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

```
[7]: df.tail()
```

[7]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
1020	59	1	1	140	221	0	1	164	1	0.0	2	0	2	1
1021	60	1	0	125	258	0	0	141	1	2.8	1	1	3	0
1022	47	1	0	110	275	0	0	118	1	1.0	1	1	2	0
1023	50	0	0	110	254	0	0	159	0	0.0	2	0	2	1
1024	54	1	0	120	188	0	1	113	0	1.4	1	1	3	0

df.columns

```
Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg','thalach',
'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'],dtype='object')
```

df.info

```
[8]: df.info

[8]: <bound method DataFrame.info of      age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  \
0     52    1   0       125   212    0        1      168      0      1.0
1     53    1   0       140   203    1        0      155      1      3.1
2     70    1   0       145   174    0        1      125      1      2.6
3     61    1   0       148   203    0        1      161      0      0.0
4     62    0   0       138   294    1        1      106      0      1.9
...   ...  ...  ..       ...   ...  ...      ...      ...    ...      ...
1020  59    1   1       140   221    0        1      164      1      0.0
1021  60    1   0       125   258    0        0      141      1      2.8
1022  47    1   0       110   275    0        0      118      1      1.0
1023  50    0   0       110   254    0        0      159      0      0.0
1024  54    1   0       120   188    0        1      113      0      1.4

      slope  ca  thal  target
0         2   2     3       0
1         0   0     3       0
2         0   0     3       0
3         2   1     3       0
4         1   3     2       0
...     ...  ..   ...     ...
1020      2   0     2       1
1021      1   1     3       0
1022      1   1     2       0
1023      2   0     2       1
1024      1   1     3       0

[1025 rows x 14 columns]>
```

df.describe()

```
[10]: df.describe()
```

[10]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	10
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.00000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	
mean	54.434146	0.695610	0.942439	131.611707	246.00000	0.149268	0.529756	149.114146	0.336585	1.071512	1.385366	0.754146	
std	9.072290	0.460373	1.029641	17.516718	51.59251	0.356527	0.527878	23.005724	0.472772	1.175053	0.617755	1.030798	
min	29.000000	0.000000	0.000000	94.000000	126.00000	0.000000	0.000000	71.000000	0.000000	0.000000	0.000000	0.000000	
25%	48.000000	0.000000	0.000000	120.000000	211.00000	0.000000	0.000000	132.000000	0.000000	0.000000	1.000000	0.000000	
50%	56.000000	1.000000	1.000000	130.000000	240.00000	0.000000	1.000000	152.000000	0.000000	0.800000	1.000000	0.000000	
75%	61.000000	1.000000	2.000000	140.000000	275.00000	0.000000	1.000000	166.000000	1.000000	1.800000	2.000000	1.000000	
max	77.000000	1.000000	3.000000	200.000000	564.00000	1.000000	2.000000	202.000000	1.000000	6.200000	2.000000	4.000000	

```
[10]: df.describe()
```

[10]:

sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
025.000000	1025.000000	1025.000000	1025.00000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
0.695610	0.942439	131.611707	246.00000	0.149268	0.529756	149.114146	0.336585	1.071512	1.385366	0.754146	2.323902	0.513171
0.460373	1.029641	17.516718	51.59251	0.356527	0.527878	23.005724	0.472772	1.175053	0.617755	1.030798	0.620660	0.500070
0.000000	0.000000	94.000000	126.00000	0.000000	0.000000	71.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
0.000000	0.000000	120.000000	211.00000	0.000000	0.000000	132.000000	0.000000	0.000000	1.000000	0.000000	2.000000	0.000000
1.000000	1.000000	130.000000	240.00000	0.000000	1.000000	152.000000	0.000000	0.800000	1.000000	0.000000	2.000000	1.000000
1.000000	2.000000	140.000000	275.00000	0.000000	1.000000	166.000000	1.000000	1.800000	2.000000	1.000000	3.000000	1.000000
1.000000	3.000000	200.000000	564.00000	1.000000	2.000000	202.000000	1.000000	6.200000	2.000000	4.000000	3.000000	1.000000

- **<u>Handle missing values :</u>**

df.isnull().sum()

df.fillna(df.mean(),inplace=True)

```
[11]: df.isnull().sum()

[11]: age        0
      sex        0
      cp         0
      trestbps   0
      chol       0
      fbs        0
      restecg    0
      thalach    0
      exang      0
      oldpeak    0
      slope      0
      ca         0
      thal       0
      target     0
      dtype: int64

[12]: df.fillna(df.mean(),inplace=True)
```

- **<u>Remove unnecessary columns</u>**

df.duplicated()

df.drop_duplicates()

df.columns

```
[13]: df.duplicated()

[13]: 0       False
      1       False
      2       False
      3       False
      4       False
              ...
      1020     True
      1021     True
      1022     True
      1023     True
      1024     True
      Length: 1025, dtype: bool
```

```
[14]: df.drop_duplicates()
```

[14]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
723	68	0	2	120	211	0	0	115	0	1.5	1	0	2	1
733	44	0	2	108	141	0	1	175	0	0.6	1	0	2	1
739	52	1	0	128	255	0	1	161	1	0.0	2	1	3	0
843	59	1	3	160	273	0	0	125	0	0.0	2	0	2	0
878	54	1	0	120	188	0	1	113	0	1.4	1	1	3	0

302 rows × 14 columns

3. Data Analysis:

- **<u>Identify outliers</u>**

  Q1=df.quantile(0.25)

  Q3=df.quantile(0.75)

  IQR=Q3-Q1

  lower_bound=Q1-1.5*IQR

  upper_bound=Q3+1.5*IQR

  outliers=((df<lower_bound) | (df>upper_bound))

  print("Outliers:",outliers.sum())

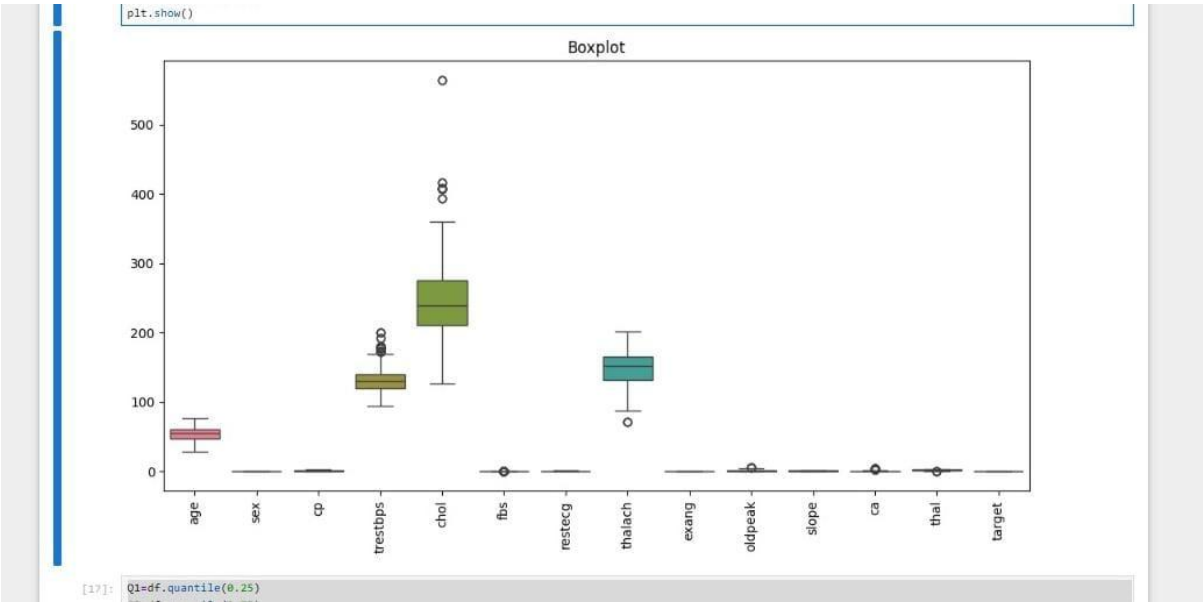
```
[17]: Q1=df.quantile(0.25)
      Q3=df.quantile(0.75)
      IQR=Q3-Q1
      lower_bound=Q1-1.5*IQR
      upper_bound=Q3+1.5*IQR
      outliers=((df<lower_bound) | (df>upper_bound))
      print("Outliers:",outliers.sum())

      Outliers: age          0
      sex            0
      cp             0
      trestbps      30
      chol          16
      fbs          153
      restecg        0
      thalach        4
      exang          0
      oldpeak        7
      slope          0
      ca            87
      thal           7
      target         0
      dtype: int64

[18]: df_clean=df[~((df<lower_bound) | (df>upper_bound)).any(axis=1)]
```

<u>Visualize outliers (boxplot)</u>

```
plt.figure(figsize=(12,6))
sns.boxplot(data=df)
plt.xticks(rotation=90)
plt.title("Boxplot")
plt.show()
```

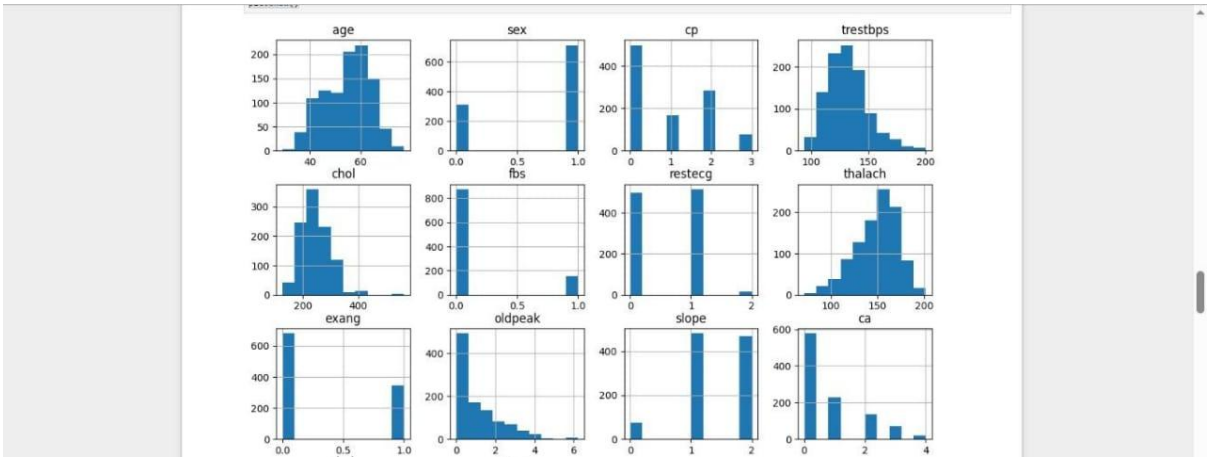


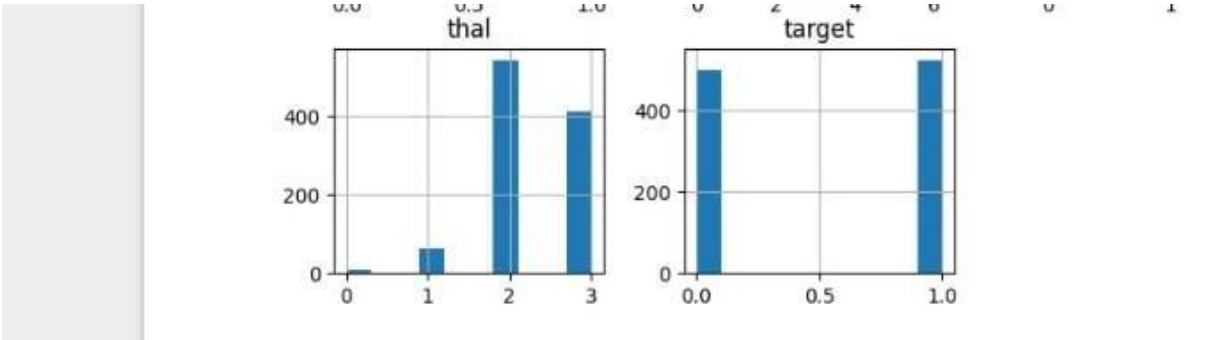
- **<u>Remove outliers</u>**

```
df_clean=df[((df<lower_bound) | (df>upper_bound)).any(axis=1)]
```

- **<u>Plot:</u>**
    - **Histogram:**

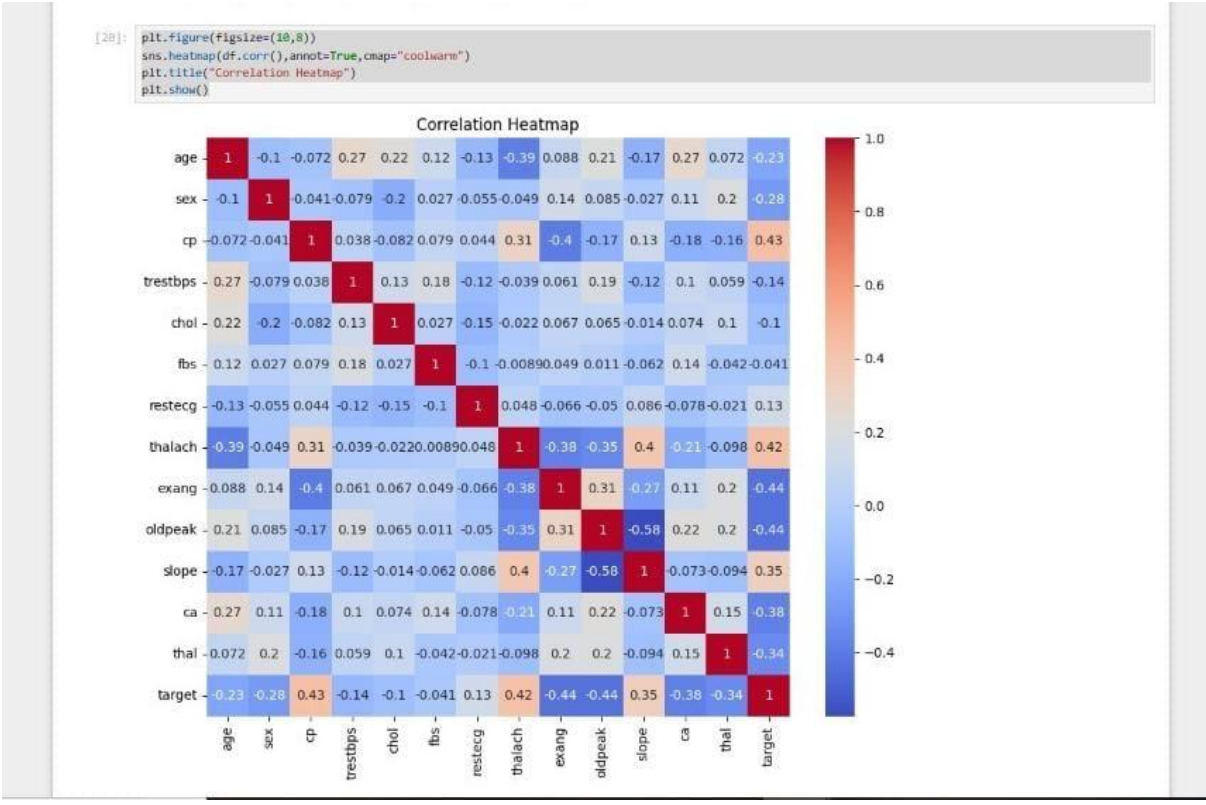
```
df.hist(figsize=(12,10))
plt.title("Histogram")
plt.show()
```





- **Correlation heatmap:**

```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(),annot=True,cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



4. Model Building

```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(),annot=True,cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

model=LogisticRegression(max_iter=1000)
model.fit(x_train,y_train)
```

```
[22]: model=LogisticRegression(max_iter=1000)
      model.fit(x_train,y_train)
```

[22]: LogisticRegression

▼ Parameters

penalty	'12'
dual	False
tol	0.0001
C	1.0
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	None
solver	'lbfgs'
max_iter	1000
multi_class	'deprecated'
verbose	0
warm_start	False
n_jobs	None
l1_ratio	None

model=LogisticRegression(max_iter=1000)

model.fit(x_train, y_train)

y_pred=model.predict(x_test)

y_pred=model.predict(x_test)

y_pred[:10]

```
[24]: y_pred=model.predict(x_test)
      y_pred[:10]

[24]: array([1, 1, 0, 1, 0, 1, 0, 0, 1, 0])
```

model=LinearRegression()

model.fit(x_train,y_train)

```
[25]: model=LinearRegression()
      model.fit(x_train,y_train)
```

```
[25]: LinearRegression
      ▼ Parameters
```

fit_intercept	True
copy_X	True
tol	1e-06
n_jobs	None
positive	False

y_pred=model.predict(x_test)

sample=pd.DataFrame([[1,0,4.5,2,0,3.1,6,9,1,0,2.3,1,0]],columns=x.columns)

prediction=model.predict(sample)

print("Prediction:",prediction)

```
Output: Prediction: [1.54357314]
```

5. Evaluation

Regression (Linear Regression)

- Evaluate using:
  - **MSE (Mean Squared Error):**

    y_pred = model.predict(x_test)

    mse = mean_squared_error(y_test, y_pred)

    print("MSE:", mse)

```
[34]: y_pred = model.predict(x_test)
      mse = mean_squared_error(y_test, y_pred)
      print("MSE:", mse)

      MSE: 0.1902439024390244
```

  - **R^2 Score:**

    y_pred = model.predict(x_test)

r2 = r2_score(y_test, y_pred)

print("R2 Score:", r2)

```
[37]: y_pred = model.predict(x_test)
      r2 = r2_score(y_test, y_pred)
      print("R2 Score:", r2)

      R2 Score: 0.23886138613861396

[ ]:
```

Classification (Logistic Regression / KNN)

- Evaluate using:

  Accuracy:

  Logistic Regression:

  model=LogisticRegression(max_iter=1000)

  model.fit(x_train, y_train)

  y_pred=model.predict(x_test)

```
[22]: model=LogisticRegression(max_iter=1000)
      model.fit(x_train,y_train)

[22]: LogisticRegression
      ▼ Parameters
```

penalty	'12'
dual	False
tol	0.0001
C	1.0
fit_intercept	True
intercept_scaling	1
class_weight	None
random_state	None
solver	'lbfgs'
max_iter	1000
multi_class	'deprecated'
verbose	0
warm_start	False
n_jobs	None
l1_ratio	None

acc=accuracy_score(y_test,y_pred)

print("Logistic Regression Accuracy:",acc)

```
[29]: acc=accuracy_score(y_test,y_pred)
      print("Logistic Regression Accuracy:",acc)

Logistic Regression Accuracy: 0.8097560975609757
```

KNN:

knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(x_train, y_train)

y_pred=knn.predict(x_test)

```
[31]: knn = KNeighborsClassifier(n_neighbors=5)
      knn.fit(x_train, y_train)

[31]: KNeighborsClassifier
      ▼ Parameters
```

n_neighbors	5
weights	'uniform'
algorithm	'auto'
leaf_size	30
p	2
metric	'minkowski'
metric_params	None
n_jobs	None

acc = accuracy_score(y_test, y_pred)

print("KNN Accuracy:",acc)

```
[32]: y_pred=knn.predict(x_test)

[33]: acc = accuracy_score(y_test, y_pred)
      print("KNN Accuracy:",acc)

KNN Accuracy: 0.7024390243902439
```

Multiple Matrics:

from sklearn.metrics import accuracy_score

```
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
```

```
print("Logistic Regression")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average='binary'))
print("Recall:", recall_score(y_test, y_pred, average='binary'))
print("F1 Score:", f1_score(y_test, y_pred,average='binary'))
```

Result:

Models achieved good accuracy.

KNN and Logistic Regression performed well.

Model can predict heart disease effectively.

Multiple Metrics.

Conclusion:

Machine learning helps in early detection.

Improves decision making in healthcare.

Future scope: Improve accuracy with more data.